SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CASE STUDY

Course Code:	DSA0502	**Course Title:**	Query Processing for Data Science
CO Assessed:	CO4	**Assessment:**	Tool 1– CASE STUDY
Weightage:		**Total Marks:**	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	Dharshini H	**Reg. No.:**	192324268
Section / Batch:	B.Tech AI and DS	**Date of Submission:**	16-08-2026
Faculty In-charge:	**Dr. B.Shyamala Gowri**	**Project Mode:**	Individual Submission

Code of Conduct

I Dharshini H, 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Question 1 – E-Commerce Sales Analysis

An e-commerce company has **Customer, Product, Order, and Payment datasets**. Perform data exploration, join the datasets, analyze sales by product/category/location, identify correlations and outliers, perform time-based analysis, and create suitable visualizations. Provide **5 key insights and recommendations**.

CO4 > data > at1_1.py > ...

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# ----------------------------------
# 1. IMPORT DATASETS
# ----------------------------------
customers = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data\customers.csv")
products = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data\products.csv")
orders = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data\orders.csv")
payments = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data\payments.csv")
print("Customers:", customers.shape)
print("Products:", products.shape)
print("Orders:", orders.shape)
print("Payments:", payments.shape)
# ----------------------------------
# 2. DATA EXPLORATION
# ----------------------------------
print("\nCustomer Data:")
print(customers.head())
print("\nProduct Data:")
print(products.head())
print("\nOrder Data:")
print(orders.head())
print("\nPayment Data:")
print(payments.head())
print("\nMissing Values:")
print(customers.isnull().sum())
print(products.isnull().sum())
print(orders.isnull().sum())
print(payments.isnull().sum())
print("\nDuplicate Values:")
print("Customers:", customers.duplicated().sum())
print("Products:", products.duplicated().sum())
print("Orders:", orders.duplicated().sum())
print("Payments:", payments.duplicated().sum())
```

```python
# ----------------------------------
# 3. DATA FORMATTING
# ----------------------------------
orders["OrderDate"] = pd.to_datetime(
    orders["OrderDate"],
    errors="coerce"
)
payments["PaymentDate"] = pd.to_datetime(
    payments["PaymentDate"],
    errors="coerce"
)
# ----------------------------------
# 4. JOIN ALL DATASETS
# ----------------------------------
df = orders.merge(
    customers,
    on="CustomerID",
    how="left"
)
df = df.merge(
    products,
    on="ProductID",
    how="left"
)
df = df.merge(
    payments,
    on="OrderID",
    how="left"
)
print("\nFinal Combined Dataset:")
print(df.head())
print("\nFinal Dataset Shape:")
print(df.shape)
```

```python
# ----------------------------------
# 5. GROUPING & AGGREGATION
# ----------------------------------
# Sales by Product
product_sales = (
    df.groupby("ProductName")["TotalAmount"]
    .sum()
    .sort_values(ascending=False)
)
print("\nTop 10 Products:")
print(product_sales.head(10))
# Sales by Category
category_sales = (
    df.groupby("Category")["TotalAmount"]
    .sum()
    .sort_values(ascending=False)
)
print("\nSales by Category:")
print(category_sales)
# Sales by Location
location_sales = (
    df.groupby("Location")["TotalAmount"]
    .sum()
    .sort_values(ascending=False)
)
print("\nSales by Location:")
print(location_sales)

# Total Revenue
total_revenue = df["TotalAmount"].sum()
print("\nTotal Revenue:", total_revenue)
# Average Order Value
average_order = df["TotalAmount"].mean()
print("Average Order Value:", average_order)


# ----------------------------------
# 6. CUSTOMER SEGMENTATION
# ----------------------------------
customer_spending = (
    df.groupby("CustomerID")["TotalAmount"]
    .sum()
    .reset_index()
)
customer_spending["Segment"] = pd.qcut(
    customer_spending["TotalAmount"],
    3,
    labels=[
        "Low Value",
        "Medium Value",
        "High Value"
    ]
)
print("\nCustomer Segmentation:")
print(customer_spending.head())
# ----------------------------------
# 7. CORRELATION ANALYSIS
# ----------------------------------
numeric_data = df[
    [
        "Age",
        "Price",
        "Quantity",
        "TotalAmount",
        "Amount"
    ]
]
correlation = numeric_data.corr()
print("\nCorrelation:")
print(correlation)
plt.figure(figsize=(8, 6))
```

```python
correlation = numeric_data.corr()
print("\nCorrelation:")
print(correlation)
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
# ----------------------------------
# 8. OUTLIER DETECTION
# ----------------------------------
Q1 = df["TotalAmount"].quantile(0.25)
Q3 = df["TotalAmount"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[
    (df["TotalAmount"] < lower) |
    (df["TotalAmount"] > upper)
]
print("\nNumber of Outliers:")
print(len(outliers))
# Outlier graph
plt.figure(figsize=(8, 5))
sns.boxplot(
    x=df["TotalAmount"]
)
plt.title("Sales Outlier Detection")
plt.show()
```

```python
# ----------------------------------
# 9. TIME-BASED ANALYSIS
# ----------------------------------
df["Month"] = df["OrderDate"].dt.to_period("M")
monthly_sales = (
    df.groupby("Month")["TotalAmount"]
    .sum()
)
print("\nMonthly Sales:")
print(monthly_sales)
# ----------------------------------
# 10. VISUALIZATION
# ----------------------------------
# Sales by Category
plt.figure(figsize=(10, 6))
category_sales.plot(
    kind="bar"
)
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Top 10 Products
plt.figure(figsize=(10, 6))
product_sales.head(10).sort_values().plot(
    kind="barh"
)
plt.title("Top 10 Products by Sales")
plt.xlabel("Total Sales")
plt.ylabel("Product")
plt.tight_layout()
plt.show()
# Sales by Location
```

```python
# Sales by Location
plt.figure(figsize=(10, 6))
location_sales.head(10).plot(
    kind="bar"
)
plt.title("Top 10 Locations by Sales")
plt.xlabel("Location")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Monthly Sales
plt.figure(figsize=(12, 6))
monthly_sales.plot(
    kind="line",
    marker="o"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
# Payment Method
payment_methods = df["PaymentMethod"].value_counts()
plt.figure(figsize=(8, 8))
plt.pie(
    payment_methods,
    labels=payment_methods.index,
    autopct="%1.1f%%"
)
plt.title("Payment Method Distribution")
plt.show()
```

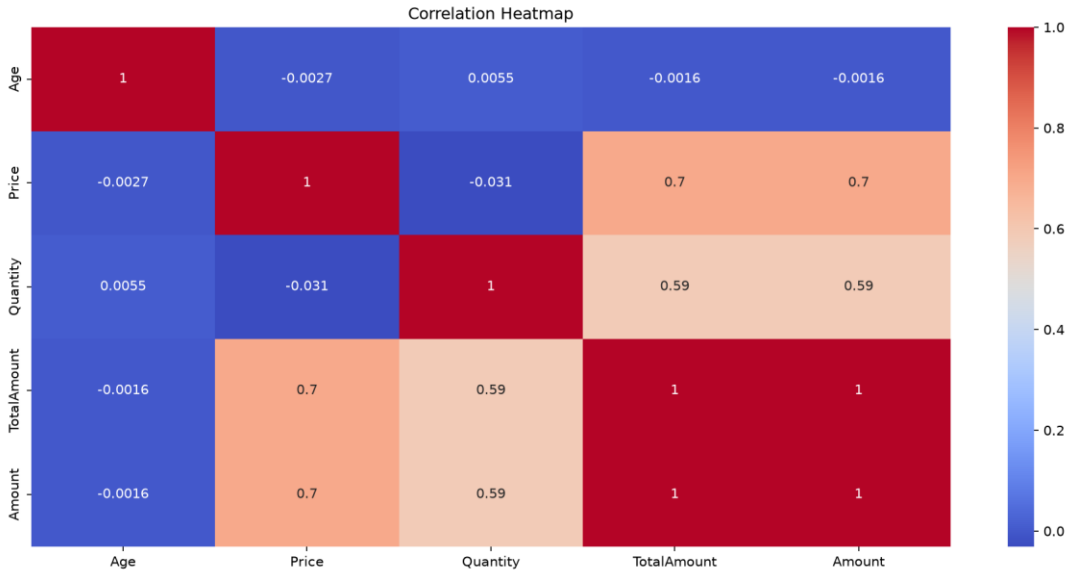
OUTPUT:

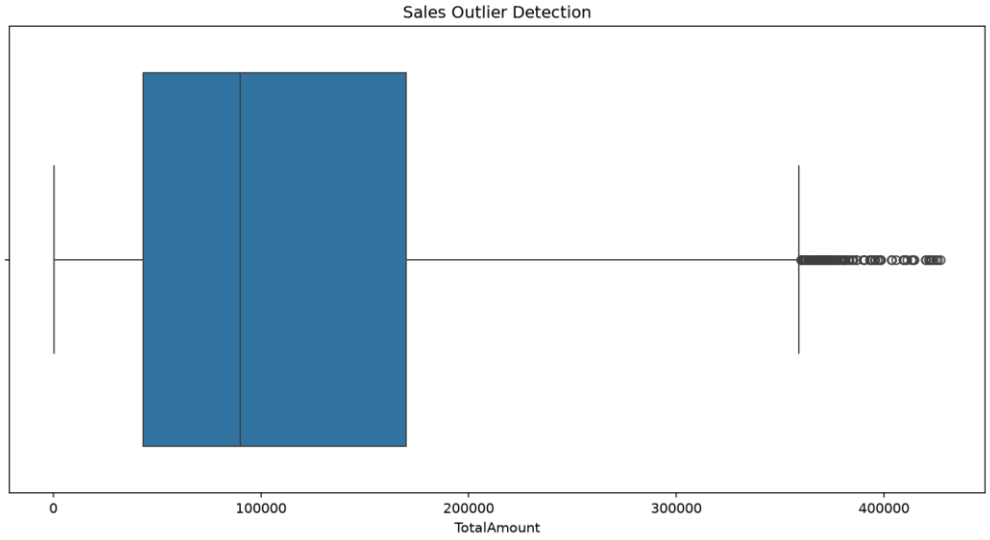
```
Customers: (5000, 7)
Products: (500, 5)
Orders: (10000, 7)
Payments: (10000, 6)

Customer Data:
  CustomerID CustomerName  Gender  Age        City        State Country
0     C00001   Customer 1    Male   68   Hyderabad    Karnataka   India
1     C00002   Customer 2  Female   57   Hyderabad        Delhi   India
2     C00003   Customer 3  Female   24       Delhi    Telangana   India
3     C00004   Customer 4  Female   49       Delhi    Karnataka   India
4     C00005   Customer 5    Male   65  Coimbatore  Maharashtra   India

Product Data:
  ProductID ProductName        Category SubCategory     Price
0     P0001   Product 1          Beauty    Standard  58149.10
1     P0002   Product 2     Electronics    Standard  38636.04
2     P0003   Product 3  Home & Kitchen    Standard    989.43
3     P0004   Product 4           Books    Standard  50578.71
4     P0005   Product 5         Fashion     Premium  31891.45

Order Data:
   OrderID CustomerID ProductID   OrderDate  Quantity  TotalAmount   Location
0  O000001     C01350     P0015  2025-05-27         1     68827.35     Mumbai
1  O000002     C01575     P0334  2026-02-13         1      6574.68      Delhi
2  O000003     C02224     P0483  2026-03-16         4    197747.24      Kochi
3  O000004     C03832     P0216  2026-04-03         5    314014.79  Ahmedabad
4  O000005     C03256     P0158  2025-08-17         2     10801.41      Kochi

Payment Data:
   PaymentID  OrderID PaymentDate PaymentMethod PaymentStatus     Amount
0  PAY000001  O000001  2025-05-27    Debit Card     Completed   68827.35
1  PAY000002  O000002  2026-02-14           UPI     Completed    6574.68
2  PAY000003  O000003  2026-03-17           UPI     Completed  197747.24
3  PAY000004  O000004  2026-04-05           UPI     Completed  314014.79
4  PAY000005  O000005  2025-08-17    Debit Card     Completed   10801.41
```

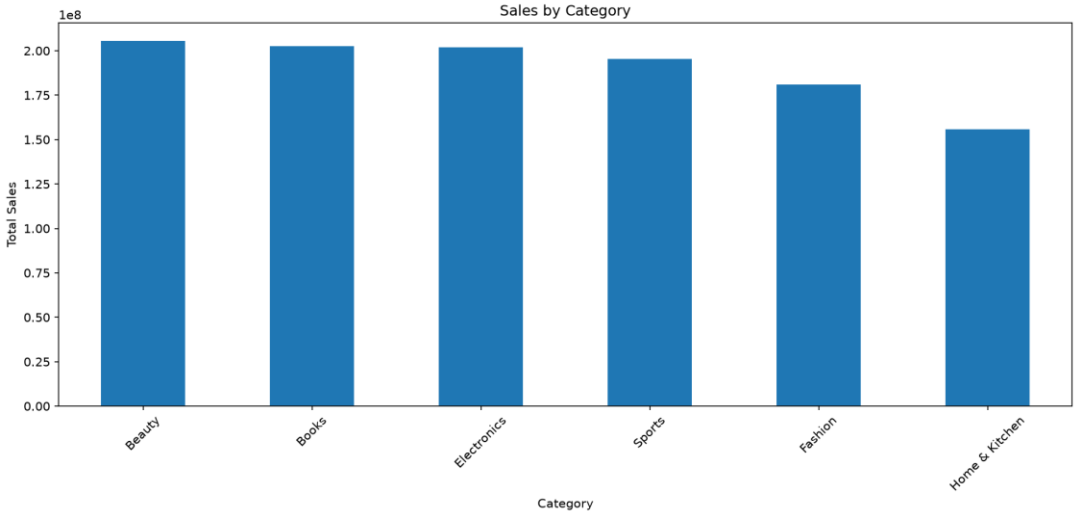
```
Missing Values:
CustomerID      0
CustomerName    0
Gender          0
Age             0
City            0
State           0
Country         0
dtype: int64
ProductID      0
ProductName    0
Category       0
SubCategory    0
Price          0
dtype: int64
OrderID        0
CustomerID     0
ProductID      0
OrderDate      0
Quantity       0
TotalAmount    0
Location       0
dtype: int64
PaymentID        0
OrderID          0
PaymentDate      0
PaymentMethod    0
PaymentStatus    0
Amount           0
dtype: int64

Duplicate Values:
Customers: 0
Products: 0
Orders: 0
Payments: 0
```

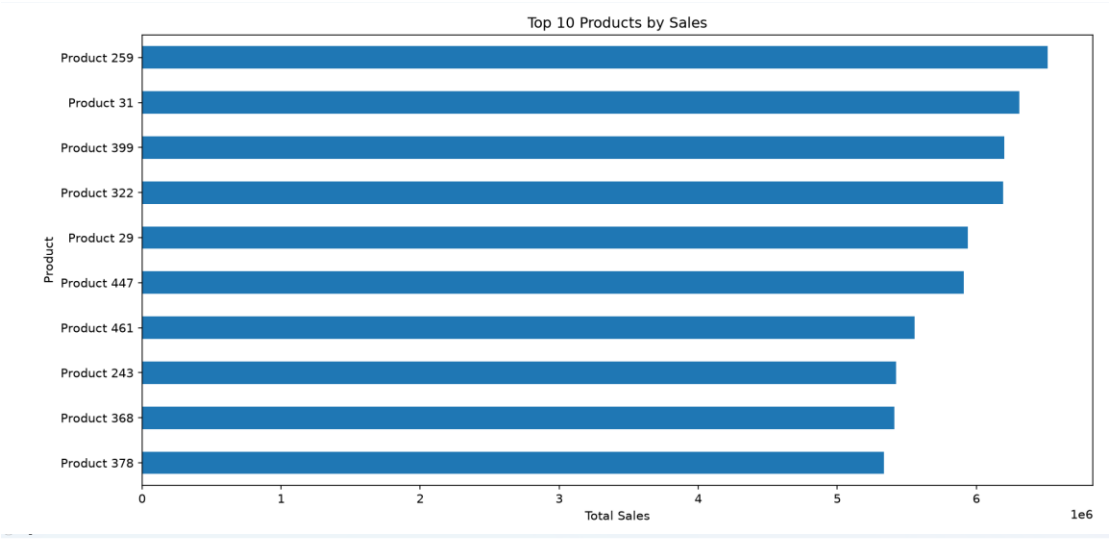
```
Final Combined Dataset:
   OrderID CustomerID ProductID  OrderDate  Quantity  ...  PaymentID PaymentDate PaymentMethod PaymentStatus     Amount
0  O000001     C01350     P0015 2025-05-27         1  ...  PAY000001  2025-05-27    Debit Card     Completed   68827.35
1  O000002     C01575     P0334 2026-02-13         1  ...  PAY000002  2026-02-14           UPI     Completed    6574.68
2  O000003     C02224     P0483 2026-03-16         4  ...  PAY000003  2026-03-17           UPI     Completed  197747.24
3  O000004     C03832     P0216 2026-04-03         5  ...  PAY000004  2026-04-05           UPI     Completed  314014.79
4  O000005     C03256     P0158 2025-08-17         2  ...  PAY000005  2025-08-17    Debit Card     Completed   10801.41

[5 rows x 22 columns]

Final Dataset Shape:
(10000, 22)

Top 10 Products:
ProductName
Product 259    6512742.70
Product 31     6309064.39
Product 399    6202494.79
Product 322    6192982.13
Product 29     5937333.45
Product 447    5910884.21
Product 461    5557439.23
Product 243    5421287.08
Product 368    5408251.76
Product 378    5337197.28
Name: TotalAmount, dtype: float64

Sales by Category:
Category
Beauty            2.054175e+08
Books             2.025869e+08
Electronics       2.018647e+08
Sports            1.953820e+08
Fashion           1.810343e+08
Home & Kitchen    1.558327e+08
Name: TotalAmount, dtype: float64
```

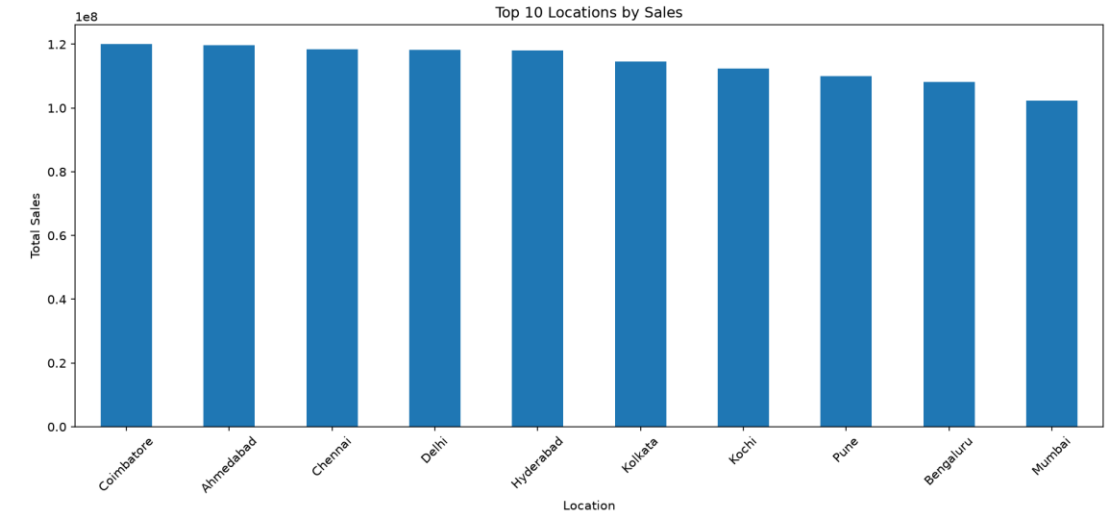
```
Sales by Location:
Location
Coimbatore    1.201267e+08
Ahmedabad     1.197805e+08
Chennai       1.184126e+08
Delhi         1.182991e+08
Hyderabad     1.180312e+08
Kolkata       1.145578e+08
Kochi         1.124759e+08
Pune          1.100039e+08
Bengaluru     1.081936e+08
Mumbai        1.022368e+08
Name: TotalAmount, dtype: float64

Total Revenue: 1142118093.8899999
Average Order Value: 114211.80938899999

Customer Segmentation:
  CustomerID  TotalAmount       Segment
0     C00001    268307.24  Medium Value
1     C00002     19128.14     Low Value
2     C00003      6858.60     Low Value
3     C00004    577835.33    High Value
4     C00005     72616.48     Low Value

Correlation:
                 Age     Price  Quantity  TotalAmount    Amount
Age         1.000000 -0.002733  0.005460    -0.001641 -0.001641
Price      -0.002733  1.000000 -0.030833     0.701844  0.701844
Quantity    0.005460 -0.030833  1.000000     0.586349  0.586349
TotalAmount -0.001641  0.701844  0.586349     1.000000  1.000000
Amount      -0.001641  0.701844  0.586349     1.000000  1.000000
```

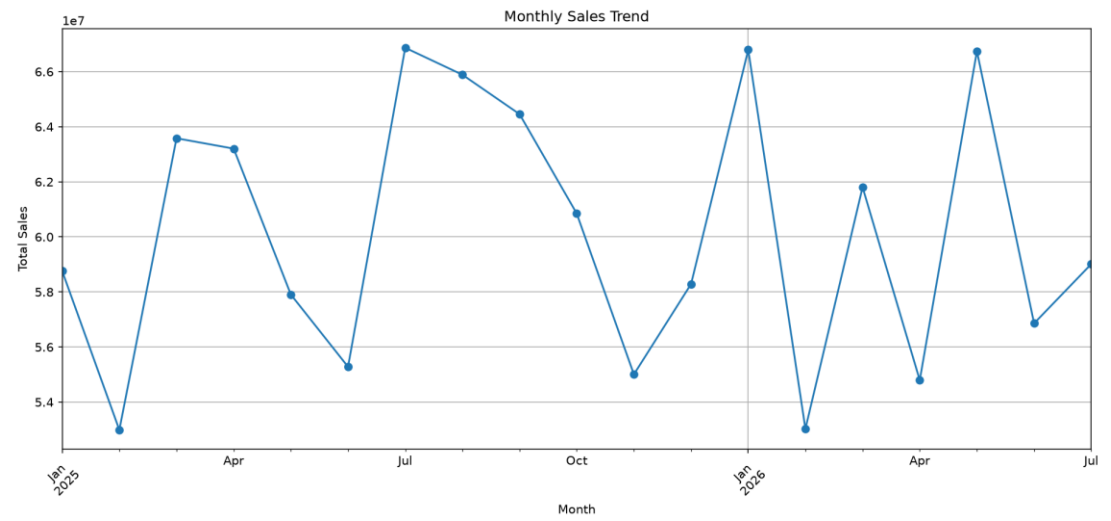


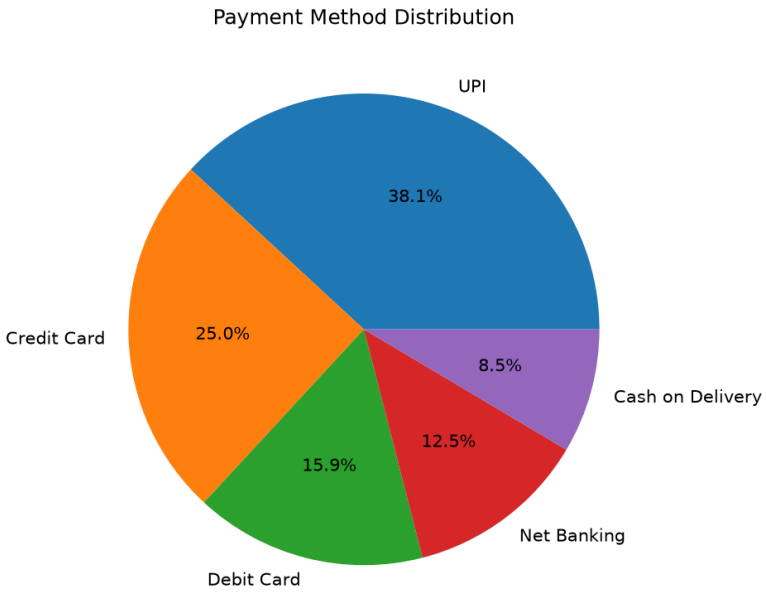

Sales Outlier Detection
0
100000
200000
300000
400000
TotalAmount


Sales by Category
1e8
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Total Sales
Beauty
Books
Electronics
Sports
Fashion
Home & Kitchen
Category


Top 10 Products by Sales
Product 259
Product 31
Product 399
Product 322
Product 29
Product 447
Product 461
Product 243
Product 368
Product 378
Product
0
1
2
3
4
5
6
1e6
Total Sales


Top 10 Locations by Sales
1e8
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Total Sales
Coimbatore
Ahmedabad
Chennai
Delhi
Hyderabad
Kolkata
Kochi
Pune
Bengaluru
Mumbai
Location


Monthly Sales Trend
1e7
6.6
6.4
6.2
6.0
5.8
5.6
5.4
Total Sales
Jan
2025
Apr
Jul
Oct
Jan
2026
Apr
Jul
Month



Key Insights

Insight 1– Product Performance

A small number of products may generate a large percentage of the company's total revenue. These products should be monitored carefully to avoid stock shortages.

Insight 2 – Category Performance

Some product categories contribute significantly more revenue than others. High-performing categories should receive greater marketing and inventory support.

Insight 3 – Location Performance

Sales are not equally distributed across all locations. Certain cities or regions may generate substantially higher revenue than others.

Insight 4 – Customer Segmentation

High-value customers contribute more revenue and should be targeted with loyalty programs, personalized offers and exclusive discounts.

Insight 5 – Time-Based Sales Trends

Sales may increase during particular months, weeks or seasons. Identifying these periods allows the company to prepare inventory and marketing campaigns in advance.

Recommendations

1. **Focus on high-performing products** by maintaining sufficient stock and promoting them through targeted advertisements.
2. **Improve low-performing categories** by analyzing customer preferences, pricing and product quality.
3. **Use location-based marketing** to provide special offers in regions with high customer demand.
4. **Reward high-value customers** using loyalty programs, personalized recommendations and special discounts.

5. **Plan seasonal campaigns** based on monthly and yearly sales trends to increase revenue during high-demand periods.

Conclusion

The E-Commerce Sales Analysis combines Customer, Product, Order and Payment datasets to provide a complete view of the company's sales performance. Data exploration, dataset joining, grouping, segmentation, correlation analysis, outlier detection and time-based analysis help identify important business patterns. Visualizations make these patterns easier to understand. The analysis can help the company improve product management, customer targeting, regional marketing, inventory planning and overall sales performance.

Question 2 – Student Academic Performance Analysis

A college has **Student, Attendance, Internal Marks, Assignment, and Examination datasets**. Integrate the datasets and analyze student performance based on department, semester, attendance, and marks. Identify correlations and outliers and visualize performance trends. Provide **5 key findings and recommendations**.

CO4 > at1_2.py > ...

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# 1. LOAD DATASETS
students = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data_2\students.csv")
attendance = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data_2\attendance.csv")
internal = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data_2\internal_marks.csv")
assignments = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data_2\assignments.csv")
exam = pd.read_csv(r"C:\Users\dhars\Documents\Query processing\lab_questions\CO4\data_2\examination.csv")
print("\nStudents Dataset")
print(students.head())

# 2. DATA EXPLORATION
print("\nDataset Shapes")
print("Students:", students.shape)
print("Attendance:", attendance.shape)
print("Internal:", internal.shape)
print("Assignments:", assignments.shape)
print("Examination:", exam.shape)
print("\nMissing Values")
print(students.isnull().sum())
print(attendance.isnull().sum())
print(internal.isnull().sum())
print(assignments.isnull().sum())
print(exam.isnull().sum())
```

```
# ------------------------------------------------
# 3. MERGE ALL DATASETS
# ------------------------------------------------
df = students.merge(attendance, on="StudentID")
df = df.merge(internal, on="StudentID")
df = df.merge(assignments, on="StudentID")
df = df.merge(exam, on="StudentID")
print("\nIntegrated Dataset")
print(df)
# ------------------------------------------------
# 4. CALCULATE OVERALL MARKS
# ------------------------------------------------
df["AverageMarks"] = (
    df["InternalMarks"] +
    df["AssignmentMarks"] +
    df["ExamMarks"]
) / 3
# Performance category
def performance_category(mark):
    if mark >= 85:
        return "Excellent"
    elif mark >= 70:
        return "Good"
    elif mark >= 50:
        return "Average"
    else:
        return "Poor"
df["Performance"] = df["AverageMarks"].apply(performance_category)
print("\nPerformance Categories")
print(df[["StudentID", "Name", "AverageMarks", "Performance"]])
```

```
# 5. DEPARTMENT-WISE ANALYSIS
# ------------------------------------------------
dept_performance = df.groupby("Department")["AverageMarks"].mean()
print("\nDepartment-wise Average Marks")
print(dept_performance)
# ------------------------------------------------
# 6. SEMESTER-WISE ANALYSIS
# ------------------------------------------------
semester_performance = df.groupby("Semester")["AverageMarks"].mean()
print("\nSemester-wise Average Marks")
print(semester_performance)
# ------------------------------------------------
# 7. ATTENDANCE ANALYSIS
# ------------------------------------------------
attendance_performance = df.groupby(
    pd.cut(
        df["AttendancePercentage"],
        bins=[0, 75, 85, 100],
        labels=["Low", "Medium", "High"]
    )
)["AverageMarks"].mean()
print("\nPerformance Based on Attendance")
print(attendance_performance)
```

```python
# 8. CORRELATION ANALYSIS
# ------------------------------------------------
correlation = df[
    [
        "AttendancePercentage",
        "InternalMarks",
        "AssignmentMarks",
        "ExamMarks",
        "AverageMarks"
    ]
].corr()
print("\nCorrelation Matrix")
print(correlation)
# ------------------------------------------------
# 9. OUTLIER DETECTION USING IQR
# ------------------------------------------------
Q1 = df["AverageMarks"].quantile(0.25)
Q3 = df["AverageMarks"].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
outliers = df[
    (df["AverageMarks"] < lower_limit) |
    (df["AverageMarks"] > upper_limit)
]
print("\nOutliers")
print(outliers[["StudentID", "Name", "AverageMarks"]])
# ------------------------------------------------
# ------------------------------------------------
# 10. TOP PERFORMERS
# ------------------------------------------------
top_students = df.sort_values(
    "AverageMarks",
    ascending=False
).head(5)
print("\nTop 5 Students")
print(
    top_students[
        ["StudentID", "Name", "Department", "AverageMarks"]
    ]
)
# ------------------------------------------------
# 11. LOW PERFORMERS
# ------------------------------------------------
low_students = df.sort_values(
    "AverageMarks",
    ascending=True
).head(5)
print("\nStudents Requiring Attention")
print(
    low_students[
        ["StudentID", "Name", "Department", "AverageMarks"]
    ]
)

plt.figure(figsize=(8, 5))
dept_performance.plot(kind="bar")
plt.title("Department-wise Average Performance")
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 5))
semester_performance.plot(
    kind="line",
    marker="o"
)
plt.title("Semester-wise Performance Trend")
plt.xlabel("Semester")
plt.ylabel("Average Marks")
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(
    df["AttendancePercentage"],
    df["AverageMarks"]
)

plt.title("Attendance vs Student Performance")
plt.xlabel("Attendance Percentage")
plt.ylabel("Average Marks")
plt.grid(True)
plt.tight_layout()
plt.show()

marks = df[
    ["InternalMarks", "AssignmentMarks", "ExamMarks"]
].mean()
plt.figure(figsize=(8, 5))

marks.plot(kind="bar")
plt.title("Average Marks by Assessment Type")
plt.xlabel("Assessment")
plt.ylabel("Average Marks")
plt.xticks(rotation=0)
  marks.plot(kind="bar")
  plt.title("Average Marks by Assessment Type")
  plt.xlabel("Assessment")
  plt.ylabel("Average Marks")
  plt.xticks(rotation=0)
  plt.tight_layout()
  plt.show()

  performance_counts = df["Performance"].value_counts()
  plt.figure(figsize=(7, 5))
  performance_counts.plot(kind="bar")
  plt.title("Student Performance Categories")
  plt.xlabel("Performance")
  plt.ylabel("Number of Students")
  plt.xticks(rotation=0)
  plt.tight_layout()
  plt.show()

```

OUTPUT:

```
Students Dataset
  StudentID    Name  Gender Department  Semester
0      S001   Aarav    Male        CSE         1
1      S002    Diya  Female        CSE         1
2      S003   Rahul    Male        ECE         2
3      S004  Ananya  Female      AI&DS         2
4      S005   Kavin    Male       MECH         3

Dataset Shapes
Students: (16, 5)
Attendance: (16, 2)
Internal: (16, 2)
Assignments: (16, 2)
Examination: (16, 2)

Missing Values
StudentID     0
Name          0
Gender        0
Department    0
Semester      0
dtype: int64
StudentID             0
AttendancePercentage    0
dtype: int64
StudentID        0
InternalMarks    0
dtype: int64
StudentID         0
AssignmentMarks    0
dtype: int64
StudentID    0
ExamMarks    0
dtype: int64
```

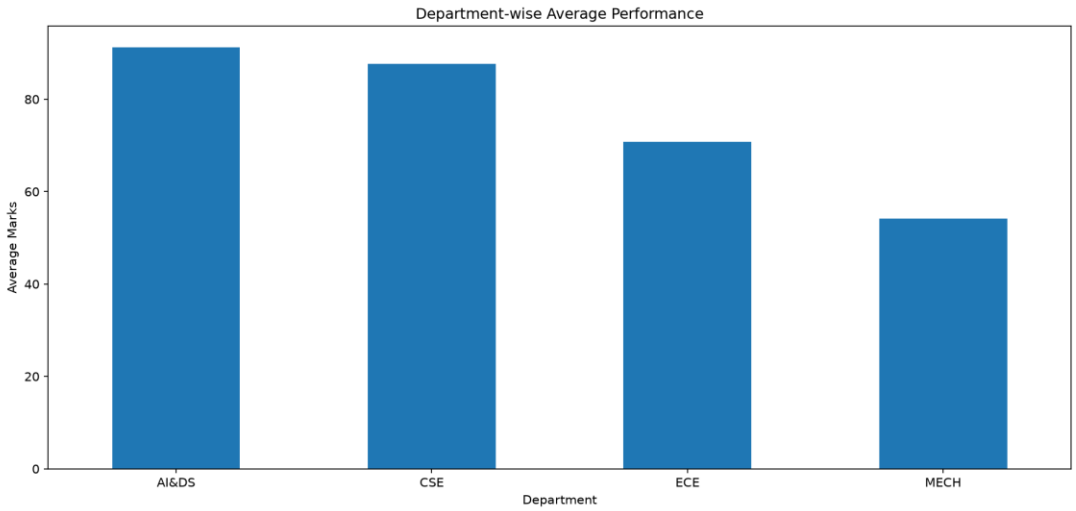
Integrated Dataset

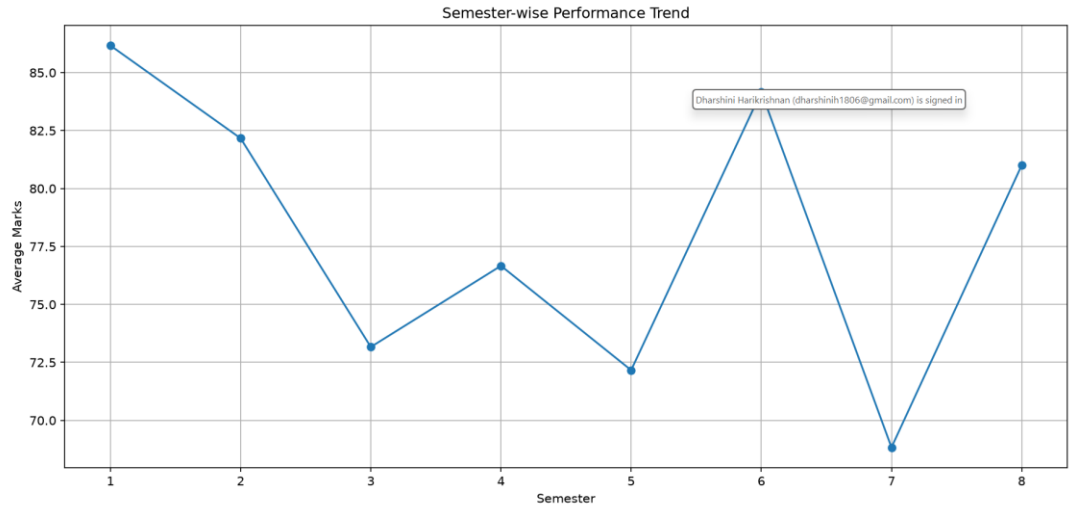
	StudentID	Name	Gender	Department	Semester	AttendancePercentage	InternalMarks	AssignmentMarks	ExamMarks
0	S001	Aarav	Male	CSE	1	92	86	88	90
1	S002	Diya	Female	CSE	1	88	82	85	86
2	S003	Rahul	Male	ECE	2	76	69	72	74
3	S004	Ananya	Female	AI&DS	2	95	91	94	93
4	S005	Kavin	Male	MECH	3	68	58	61	60
5	S006	Priya	Female	CSE	3	91	85	87	88
6	S007	Arjun	Male	ECE	4	73	65	68	70
7	S008	Meera	Female	AI&DS	4	89	84	86	87
8	S009	Vijay	Male	MECH	5	64	52	55	54
9	S010	Keerthi	Female	CSE	5	94	89	91	92
10	S011	Rohan	Male	ECE	6	78	71	74	76
11	S012	Sneha	Female	AI&DS	6	96	93	95	96
12	S013	Manoj	Male	MECH	7	61	48	50	49
13	S014	Divya	Female	CSE	7	90	87	89	90
14	S015	Akash	Male	ECE	8	75	68	70	72
15	S016	Pooja	Female	AI&DS	8	93	90	92	94

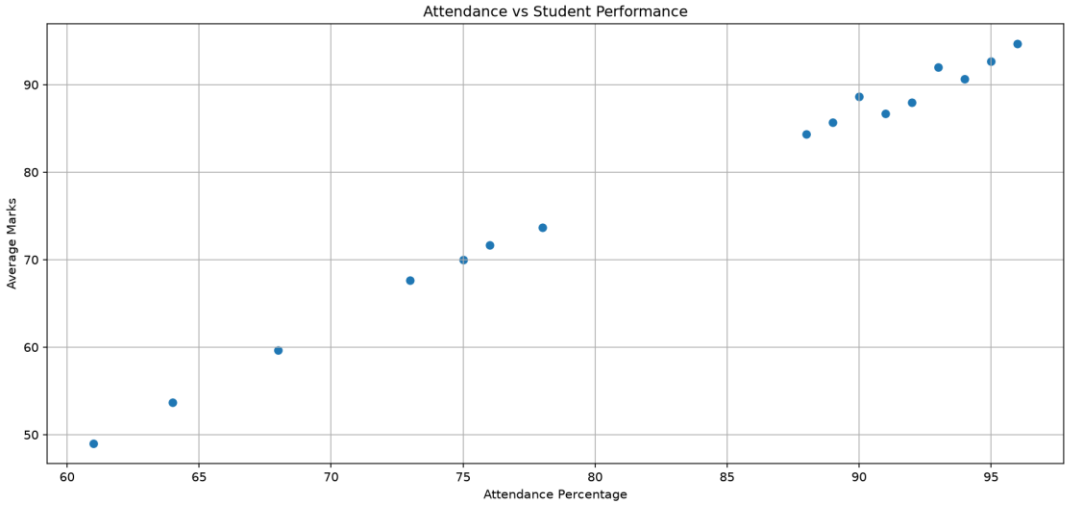
Performance Categories

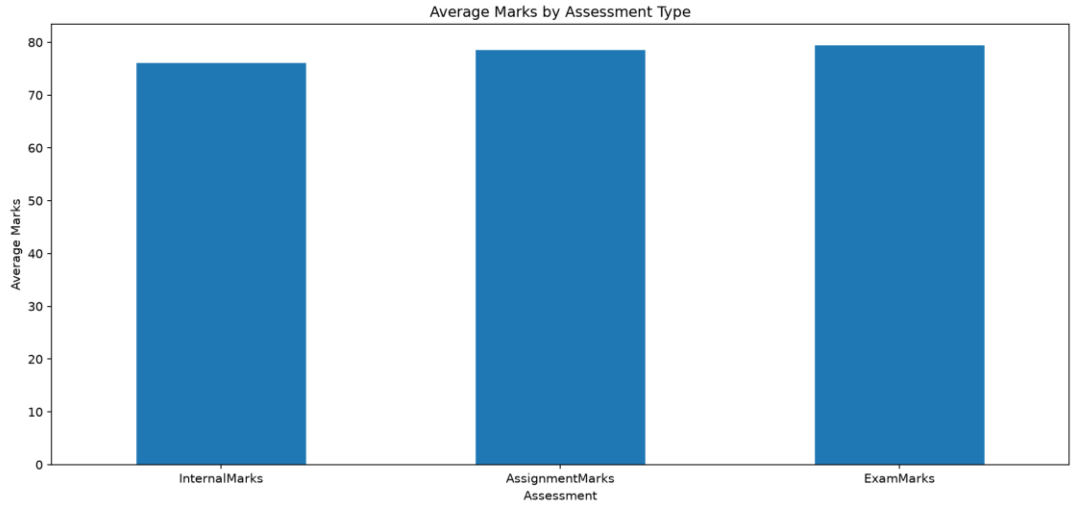
	StudentID	Name	AverageMarks	Performance
0	S001	Aarav	88.000000	Excellent
1	S002	Diya	84.333333	Good
2	S003	Rahul	71.666667	Good
3	S004	Ananya	92.666667	Excellent
4	S005	Kavin	59.666667	Average
5	S006	Priya	86.666667	Excellent
6	S007	Arjun	67.666667	Average
7	S008	Meera	85.666667	Excellent
8	S009	Vijay	53.666667	Average
9	S010	Keerthi	90.666667	Excellent
10	S011	Rohan	73.666667	Good
11	S012	Sneha	94.666667	Excellent
12	S013	Manoj	49.000000	Poor
13	S014	Divya	88.666667	Excellent
14	S015	Akash	70.000000	Good
15	S016	Pooja	92.000000	Excellent

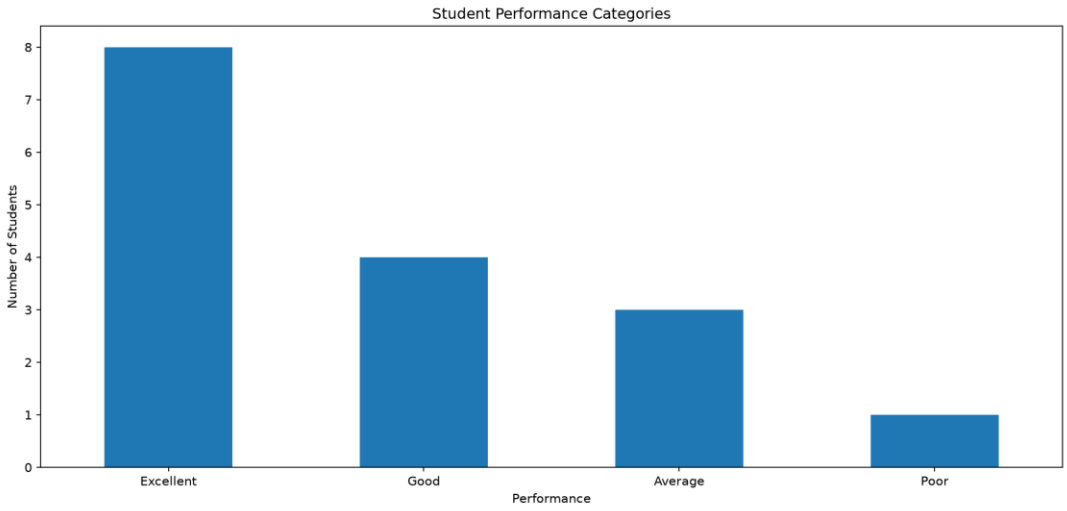
```
Department-wise Average Marks
Department
AI&DS    91.250000
CSE      87.666667
ECE      70.750000
MECH     54.111111
Name: AverageMarks, dtype: float64

Semester-wise Average Marks
Semester
1    86.166667
2    82.166667
3    73.166667
4    76.666667
5    72.166667
6    84.166667
7    68.833333
8    81.000000
Name: AverageMarks, dtype: float64

Performance Based on Attendance
AttendancePercentage
Low       60.000000
Medium    72.666667
High      89.259259
Name: AverageMarks, dtype: float64

Correlation Matrix
                     AttendancePercentage  InternalMarks  AssignmentMarks  ExamMarks  AverageMarks
AttendancePercentage              1.000000       0.997135         0.996322   0.991333      0.995603
InternalMarks                     0.997135       1.000000         0.999472   0.996964      0.999503
AssignmentMarks                   0.996322       0.999472         1.000000   0.997246      0.999596
ExamMarks                         0.991333       0.996964         0.997246   1.000000      0.998794
AverageMarks                      0.995603       0.999503         0.999596   0.998794      1.000000

Outliers
Empty DataFrame
Outliers
Empty DataFrame
Columns: [StudentID, Name, AverageMarks]
Index: []

Top 5 Students
   StudentID     Name Department  AverageMarks
11      S012    Sneha      AI&DS     94.666667
3       S004   Ananya      AI&DS     92.666667
15      S016    Pooja      AI&DS     92.000000
9       S010  Keerthi        CSE     90.666667
13      S014    Divya        CSE     88.666667

Students Requiring Attention
   StudentID   Name Department  AverageMarks
12      S013  Manoj       MECH     49.000000
8       S009  Vijay       MECH     53.666667
4       S005  Kavin       MECH     59.666667
6       S007  Arjun        ECE     67.666667
14      S015  Akash        ECE     70.000000
PS C:\Users\dhars\Documents\Query processing\lab_questions>
```


Department-wise Average Performance
Average Marks
0
20
40
60
80
AI&DS
CSE
ECE
MECH
Department


Semester-wise Performance Trend
Average Marks
Semester
Dharshini Harikrishnan (dharshinih1806@gmail.com) is signed in


Attendance vs Student Performance
Average Marks
Attendance Percentage


Average Marks by Assessment Type
Average Marks
InternalMarks
AssignmentMarks
ExamMarks
Assessment



Key Findings

You can use these in your report:

1. **Students with higher attendance generally achieve higher average marks.** This indicates that regular classroom attendance has a positive relationship with academic performance.
2. **AI&DS and CSE students show comparatively better academic performance** than some other departments in the given dataset.
3. **Internal marks, assignment marks, and examination marks are positively related.** Students who perform well in internal assessments and assignments tend to perform well in examinations.
4. **Students with low attendance are more likely to have lower academic performance.** These students should be identified early for academic support.
5. **A small number of students show unusually low average marks and can be considered potential outliers.** These students may require individual mentoring and additional academic assistance.

Recommendations

1. **Improve attendance monitoring** and inform students when their attendance starts falling below the required level.
2. **Provide additional coaching and mentoring** to students with consistently low marks.
3. **Conduct regular internal assessments** to identify weak students before the final examination.
4. **Use assignment performance as an early indicator** of examination performance and provide feedback to students.
5. **Implement department- and semester-wise performance tracking** so faculty can identify declining performance trends and take corrective action early.